Aula 7: Muitas Formas - Polimorfismo de Sobrescrita



Projeto: FiapRide

Foco: @Override, Contratos e Comportamentos Específicos

🎯 Objetivo da Aula

Imagine que o sistema precisa exibir para o motorista quantos quilômetros ele ainda consegue rodar com o tanque atual. O comando do sistema é único: "Calcule a Autonomia".

Porém, a lógica matemática para um **Carro** (que faz 10 km por litro) é totalmente diferente da lógica de uma **Moto** (que é mais leve e faz 35 km por litro).

Hoje você vai aprender o **Polimorfismo de Sobrescrita**. Vamos definir um método padrão na classe mãe e permitir que cada classe filha "reescreva" esse método com sua própria regra de negócio.

Conceitos Chave:

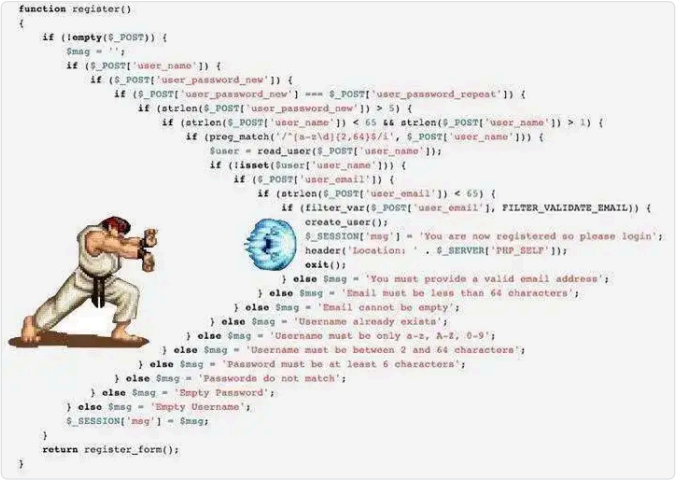
- **Polimorfismo:** "Poli" (muitos) + "Morfismo" (formas). Uma mesma chamada de método que se comporta de formas diferentes.
- **Sobrescrita (@Override):** Quando a classe filha ignora o comportamento padrão da mãe e impõe o seu próprio.
- **Contrato:** A assinatura do método (nome, retorno e parâmetros) deve ser idêntica em todas as classes para que a mágica funcione.

📖 A Teoria: O mesmo botão, ações diferentes

Pense no botão "Play" de um controle remoto universal.

- Se você aponta para o **Som**, o botão Play inicia uma música.
- Se você aponta para a **TV**, o botão Play inicia um filme.

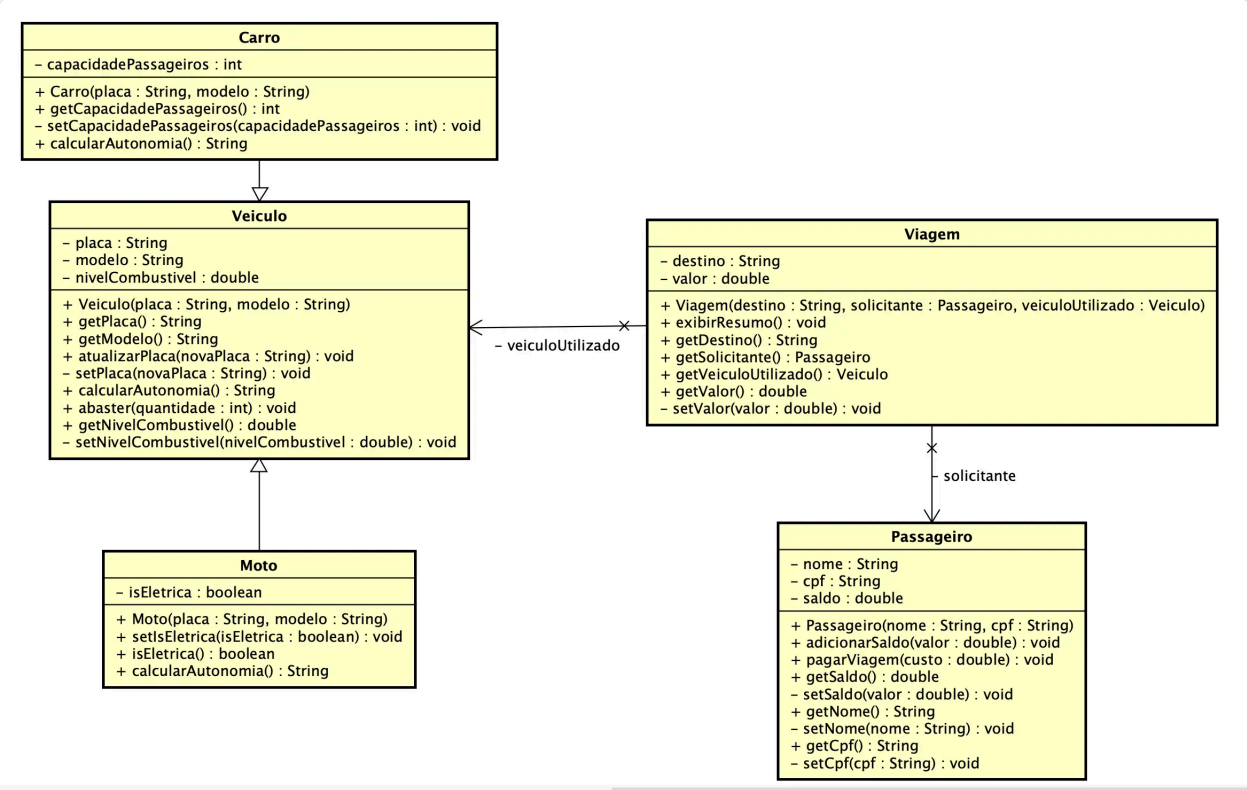
O botão é o mesmo, mas o aparelho que recebe o comando decide o que fazer. No Java, usamos a anotação `@Override` para avisar ao compilador: *"Ei, eu sei que esse método existe na mãe, mas eu sou uma Moto e quero calcular isso do meu jeito!"*.



🛠️ Parte 1: Modelagem Visual (Astah)

Vamos atualizar as classes `Veiculo`, `Carro` e `Moto`.

1. Abra o **Astah**.
2. Na classe **Veiculo**, adicione o método: `+ calcularAutonomia(): String`
3. Nas classes **Carro** e **Moto**, adicione exatamente o **MESMO** método: `+ calcularAutonomia(): String`



- No UML, quando o método aparece na filha com a mesma assinatura da mãe, estamos representando a sobrescrita.

💻 Parte 2: O Desafio (Mão na Massa)

Passo 1: O comportamento base na classe `Veiculo`

A classe mãe dita a regra (o contrato), mas a resposta dela é genérica.

```
package br.com.fiapride.model;

public class Veiculo {

    private String placa;
    private String modelo;
    private double nivelCombustivel;

    public Veiculo(String placa, String modelo) {
        this.setPlaca(placa);
        this.modelo = modelo;
    }

    public String getPlaca() {
        return this.placa;
    }
```

```
    public String getModelo() {
        return this.modelo;
    }

    public void atualizarPlaca(String novaPlaca) {
        System.out.println("🔄 Solicitada atualização de placa no Detran para o veículo " + this.mod
elo + "...");
        this.setPlaca(novaPlaca);
    }

    private void setPlaca(String novaPlaca) {
        if (novaPlaca != null && !novaPlaca.trim().isEmpty()) {
            this.placa = novaPlaca;
        } else {
            System.out.println("❌ Erro de Validação: A placa informada é inválida!");
        }
    }

    public String calcularAutonomia() {
        return "Autonomia não definida para um veículo genérico.";
    }

    public void abaster(int quantidade) {
        this.setNivelCombustivel( this.getNivelCombustivel() + quantidade);
    }

    public double getNivelCombustivel() {
        return nivelCombustivel;
    }

    private void setNivelCombustivel(double nivelCombustivel) {
        this.nivelCombustivel = nivelCombustivel;
    }
}
```

Passo 2: A Sobrescrita na classe `Carro`

O carro gasta mais combustível. Vamos supor que ele faça **10 km/l**.

```
package br.com.fiapride.model;

public class Carro extends Veiculo {

    private int capacidadePassageiros;

    public Carro (String placa, String modelo) {
        super(placa, modelo);
        this.setCapacidadePassageiros(4);
    }
```

```
    public int getCapacidadePassageiros() {
        return this.capacidadePassageiros;
    }

    private void setCapacidadePassageiros(int capacidadePassageiros) {
        this.capacidadePassageiros = capacidadePassageiros;
    }

    @Override
    // Anotação obrigatória (Boas Práticas!)
    public String calcularAutonomia() {
        double kmRestante = this.getNivelCombustivel() * 10.0;
        return "Autonomia: " + kmRestante + " km (Consumo de 10 km/l).";
    }
}
```

Passo 3: A Sobrescrita na classe `Moto`

A moto é econômica. Vamos supor que ela faça **35 km/l**.

```
package br.com.fiapride.model;

public class Moto extends Veiculo {

    private boolean isEletrica;

    public Moto(String placa, String modelo) {
        super(placa, modelo);
    }

    public void setIsEletrica (boolean isEletrica) {
        this.isEletrica = isEletrica;
    }

    public boolean isEletrica() {
        return this.isEletrica;
    }

    @Override
    public String calcularAutonomia() {
        double kmRestante = this.getNivelCombustivel() * 35.0;
        return "Autonomia: " + kmRestante + " km (Consumo de 35 km/l).";
    }
}
```

🧪 Parte 3: O Teste Polimórfico (`SistemaPrincipal`)

Agora vem a recompensa do seu trabalho. Vamos criar uma lista de veículos "misturados" e dar **o mesmo comando** para todos.

```java
package br.com.fiapride.main;
import br.com.fiapride.model.*;
import java.util.ArrayList;
import java.util.List;

public class SistemaPrincipal {
    public static void main(String[] args) {

        // 1. Criamos uma lista da SUPERCLASSE. Ela aceita qualquer herdeiro!
        List<Veiculo> frota = new ArrayList<>();
        // 2. Colocamos 50 litros em ambos os veículos
        frota.add(new Carro("ABC-1234", "Honda Civic"));
        frota.add(new Moto("DEF-5678", "Honda CG 160"));
        System.out.println("--- ⛽ RELATÓRIO DE AUTONOMIA DA FROTA ---");
        // 3. A MÁGICA: O mesmo comando produz resultados diferentes!
        for (Veiculo veiculo : frota) {
            System.out.println("Veículo: " + veiculo.getModelo());
            // O Java é inteligente: se for Carro, usa a conta do carro. Se for Moto, usa a da moto.
            veiculo.abaster(50);
            System.out.println(veiculo.calcularAutonomia());
            System.out.println("----------------------------------------");
        }
    }
}
```

⚡ Microdesafio Pessoal: O Poder das Múltiplas Formas

Agora que você viu a mágica do Polimorfismo acontecer no FiapRide, é hora de aplicar esse conceito no **seu** projeto pessoal (seja ele um Pet, um Smartphone, um Eletrodoméstico, etc.).

Sua Missão:

1. **Na Superclasse (Mãe):** Escolha ou crie um método de ação genérico que faça sentido para toda a sua hierarquia. *(Ex: `emitirSom()` para um Pet, ou `calcularTaxaSeguro()` para um Aparelho Eletrônico)*. A resposta inicial deve ser genérica.
2. **Nas Subclasses (Filhas):** Vá em pelo menos duas classes filhas e sobrescreva esse método utilizando a anotação `@Override`. Lembre-se: o contrato (assinatura do método) não pode mudar!
3. **A Regra de Negócio:** Imponha a "personalidade" de cada objeto. Cada filha deve ter um retorno ou cálculo diferente. *(Ex: O `Cachorro` retorna "Au Au!", o `Gato` retorna "Miau!". O `Smartphone` calcula a taxa baseada em 15% do valor, o `Notebook` calcula com base em 20%).*
4. **O Teste de Fogo:** Vá para a sua classe principal de testes (`main`). Crie uma `List<>` tipada com a sua Superclasse. Adicione seus objetos específicos nela e crie um laço `for` chamando o método.

Se ao rodar o código o mesmo comando gerar resultados completamente diferentes no console para cada item da lista, parabéns: você dominou o Polimorfismo!

🧼 Checklist: Clean Code (O Contrato é Sagrado)

- [ ] **Mesma Assinatura:** Para que o polimorfismo funcione, você não pode mudar o retorno ou os parâmetros. Se a mãe retorna `String`, a filha DEVE retornar `String`. Se você mudar, o Java entende que você está criando um método novo, e não sobrescrevendo.
- [ ] **Anotação @Override:** Sempre use. Se você errar uma letra no nome do método (ex: `calcularAutonomiaS`), a anotação força o compilador a te avisar do erro.
- [ ] **Sem `if` para descobrir o tipo:** O pior crime em Orientação a Objetos é fazer `if (veiculo é moto) { conta 1 } else { conta 2 }`. Deixe que o próprio objeto (a classe) cuide da sua regra internamente!

🧠 Pergunta de Reflexão

"No nosso loop `for (Veiculo veiculo : frota)`, a variável `veiculo` é do tipo genérico `Veiculo`. Se esquecêssemos de criar o método `calcularAutonomia()` lá na classe mãe `Veiculo`, nós conseguiríamos chamá-lo dentro do loop, mesmo sabendo que ele existe dentro do `Carro` e da `Moto`? Por que o contrato precisa existir na base da hierarquia?"

📤 Finalizando (GitHub)

```
git add .
git commit -m "feat: aplica polimorfismo de sobrescrita"
git push origin main
```

Bons estudos!

